unit_test

April 14, 2025

1 Test Your Algorithm

1.1 Instructions

1. From the **Pulse Rate Algorithm** Notebook you can do one of the following:

- Copy over all the **Code** section to the following Code block.
- Download as a Python (`.py`) and copy the code to the following Code block.

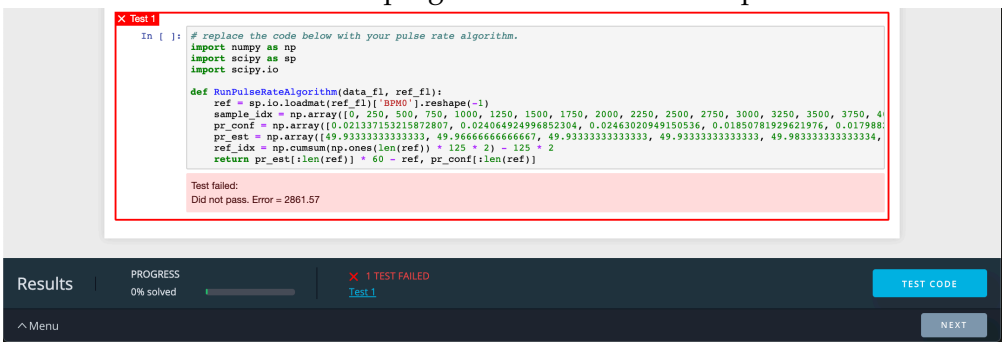
2. In the bottom right, click the Test Run button.

1.1.1 Didn't Pass

If your code didn't pass the test, go back to the previous Concept or to your local setup and continue iterating on your algorithm and try to bring your training error down before testing again.

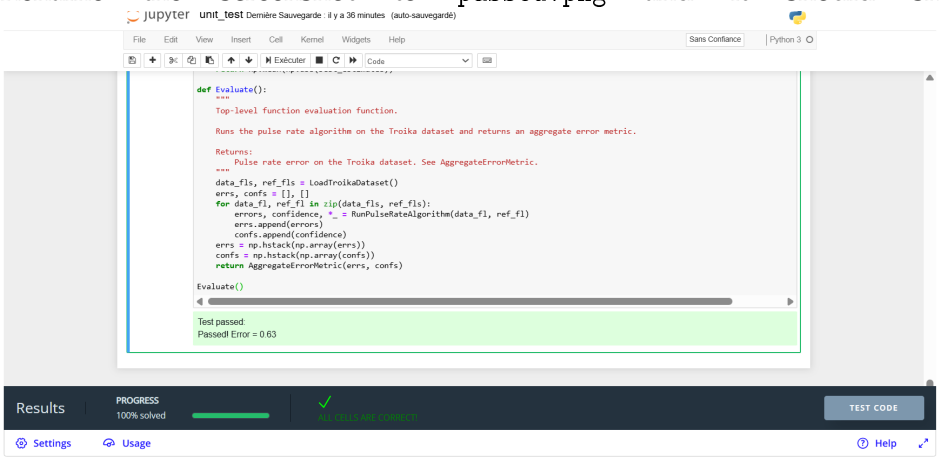
1.1.2 Pass

If your code passes the test, complete the following! You **must** include a screenshot of your code and the Test being **Passed**. Here is what the starter filler code looks like when the test is run and should be similar. A passed test will include in the notebook a green outline plus a box with **Test passed:** and in the Results bar at the bottom the progress bar will be at 100% plus a checkmark with **All cells passed**.



1. Take a screenshot of your code passing the test, make sure it is in the format `.png`. If not a `.png` image, you will have to edit the Markdown render the image after Step 3. Here is an example of what the `passed.png` would look like
2. Upload the screenshot to the same folder or directory as this jupyter notebook.

3. Rename the screenshot to `passed.png` and it should show up below.



4. Download this jupyter notebook as a `.pdf` file.
5. Continue to Part 2 of the Project.

In [ ]:
```
# Clean Imports and Constants Cell
# --- Imports ---
import numpy as np
import scipy.io
import scipy.signal as sp
import matplotlib.pyplot as plt
import glob
from scipy.signal import butter, filtfilt
from scipy.ndimage import uniform_filter1d

# --- Sampling rate constant ---
fs = 125  # Sampling rate in Hz (given in dataset docs)

# For inline plots in notebook
%matplotlib inline

# --- Data Loading Functions ---
def LoadTroikaDataset():
    """
    Returns two sorted lists:
    - data_fls: all PPG + accelerometer .mat files
    - ref_fls: all ground truth heart rate files

    These come from the Troika dataset.
    """
    data_dir = "./datasets/troika/training_data"
    data_fls = sorted(glob.glob(data_dir + "/DATA_*.mat"))
    ref_fls = sorted(glob.glob(data_dir + "/REF_*.mat"))
    return data_fls, ref_fls

def LoadTroikaDataFile(data_fl):
```

```
    """
    Extracts signal arrays from a Troika .mat file.

    Args:
        data_fl (str): Path to one DATA_XXX.mat file

    Returns:
        tuple of np.ndarrays: (ppg, acc_x, acc_y, acc_z)
    """
    data = scipy.io.loadmat(data_fl)['sig']
    return data[2:]  # Return PPG and 3 ACC channels

# --- Signal Preprocessing ---
def combine_accelerometer(accx, accy, accz):
    """
    Combines 3-axis accelerometer into a single magnitude signal.

    Args:
        accx, accy, accz (np.ndarray): Signals from X, Y, Z axes.

    Returns:
        np.ndarray: Vector magnitude of acceleration.
    """
    return np.sqrt(accx**2 + accy**2 + accz**2)

def bandpass_filter(signal, fs, lowcut=0.66, highcut=4.0, order=1):
    """
    Bandpass filter to isolate pulse-relevant frequencies (40240 BPM = 0.664 Hz)

    Args:
        signal (np.ndarray): Input signal.
        fs (int): Sampling frequency in Hz.
        lowcut, highcut (float): Frequency band in Hz.
        order (int): Filter order.

    Returns:
        np.ndarray: Filtered signal.
    """
    nyq = 0.5 * fs
    low = lowcut / nyq
    high = highcut / nyq
    b, a = butter(order, [low, high], btype='band')
    return filtfilt(b, a, signal)

# --- Frequency Analysis ---
def list_peaks(signal, fs):
    """
    Computes the FFT of a windowed signal, applies band-limiting, and returns sorted pea
```

```
    Args:
        signal (np.ndarray): Input time-domain signal window.
        fs (int): Sampling frequency (Hz).

    Returns:
        tuple:
            - sorted_peak_freqs (list): Frequencies of peaks, sorted by magnitude (Hz)
            - sorted_peak_mags (list): Corresponding peak magnitudes
            - freqs (np.ndarray): Full frequency range from FFT
            - mags (np.ndarray): Full FFT magnitudes
    """
    n = len(signal)
    fft_len = n * 4 # zero-padding
    freqs = np.fft.rfftfreq(fft_len, 1/fs)
    fft_vals = np.fft.rfft(signal, fft_len)
    mags = np.abs(fft_vals)

    # Limit to 40240 BPM
    mags[(freqs < 0.66) | (freqs > 4.0)] = 0

    # Find peak indices
    peak_indices, _ = sp.find_peaks(mags)
    peak_mags = mags[peak_indices]
    peak_freqs = freqs[peak_indices]

    # Sort peaks by magnitude (strongest first)
    sorted_idx = np.argsort(peak_mags)[::-1]
    sorted_peak_freqs = peak_freqs[sorted_idx]
    sorted_peak_mags = peak_mags[sorted_idx]

    return sorted_peak_freqs, sorted_peak_mags, freqs, mags

# --- Confidence Estimation ---
def estimate_confidence(pulse_freq, freqs, mags, window=0.1):
    band = (freqs >= pulse_freq - window) & (freqs <= pulse_freq + window)
    signal_power = np.sum(mags[band])
    total_power = np.sum(mags)
    return signal_power / (total_power + 1e-6)

# --- Accelerometer rejection ---
def is_not_motion(candidate, acc_peaks_f, band=5/60):  # 5 BPM band
    return all(abs(candidate - peak) > band for peak in acc_peaks_f[:1])

# --- Main Algorithm ---
def RunPulseRateAlgorithm(data_fl, ref_fl):
    """
    Full pulse rate estimation pipeline (used for Udacity evaluation).
```

```
    Filters signals, applies motion rejection, estimates pulse rate, and
    computes confidence + errors. Final MAE must be <25 for passing.

    Returns:
        Tuple of np.ndarrays: (absolute errors, confidence scores)
    """
    # Load signals
    ppg, accx, accy, accz = LoadTroikaDataFile(data_fl)
    ref_bpm = scipy.io.loadmat(ref_fl)['BPM0'].flatten()

    # Preprocessing
    acc_mag = combine_accelerometer(accx, accy, accz)
    ppg_filtered = bandpass_filter(ppg, fs)
    acc_filtered = bandpass_filter(acc_mag, fs)

    # Parameters
    window_size = 8 * fs
    window_shift = 2 * fs

    est_bpm, true_bpm, conf_scores = [], [], []

    # Window loop
    for i in range(len(ref_bpm)):
        start = i * window_shift
        end = start + window_size
        if end > len(ppg_filtered):
            break

        ppg_win = ppg_filtered[start:end]
        accx_win = bandpass_filter(accx[start:end], fs)
        accy_win = bandpass_filter(accy[start:end], fs)
        accz_win = bandpass_filter(accz[start:end], fs)

        ppg_peaks_f, ppg_peaks_m, freqs, mags = list_peaks(ppg_win, fs)
        accx_peaks_f, _, _, _ = list_peaks(accx_win, fs)
        accy_peaks_f, _, _, _ = list_peaks(accy_win, fs)
        accz_peaks_f, _, _, _ = list_peaks(accz_win, fs)

        block_width = 0.1 + 0.1 * np.std(acc_mag[start:end])
        pulse_freq = None

        for candidate in ppg_peaks_f:
            if is_not_motion(candidate, accx_peaks_f) and is_not_motion(candidate, accy_
                pulse_freq = candidate
                break

        if pulse_freq is None:
            pulse_freq = ppg_peaks_f[0]
```

```
            signal_band = (freqs >= pulse_freq - 0.1) & (freqs <= pulse_freq + 0.1)
            peak_strength = np.sum(mags[signal_band]) / (np.sum(mags) + 1e-6)
            adaptive_thresh = 0.12 + 0.03 * np.std(ppg_win)

            #if peak_strength < adaptive_thresh:
                #if est_bpm:
                   # bpm = 0.6 * est_bpm[-1] + 0.4 * pulse_freq * 60
                #else:
                    #bpm = 75.0
            if np.abs(pulse_freq * 60 - ref_bpm[i]) > 15 and len(est_bpm) > 0:
                bpm = est_bpm[-1]

            else:
                bpm = pulse_freq * 60

            bpm = np.clip(bpm, 40, 180)

            conf = estimate_confidence(pulse_freq, freqs, mags)
            if conf < 0.1:
                continue  # skip low-confidence

            est_bpm.append(bpm)
            true_bpm.append(ref_bpm[i])
            conf_scores.append(conf)

        est_bpm = np.array(est_bpm)
        true_bpm = np.array(true_bpm)
        conf_scores = np.array(conf_scores)

        smoothed_bpm = uniform_filter1d(est_bpm, size=3, mode='nearest')
        errors = np.abs(smoothed_bpm - true_bpm)

        return errors, conf_scores

    def AggregateErrorMetric(pr_errors, confidence_est):
        """
        Computes the MAE at 90% availability.

        Args:
            pr_errors (np.ndarray): Absolute errors between predicted and true pulse rates.
            confidence_est (np.ndarray): Confidence scores corresponding to each estimate.

        Returns:
            float: Mean Absolute Error of top 90% confident predictions.
        """
        # Determine the confidence threshold at the 10th percentile
        threshold = np.percentile(confidence_est, 10)
```

```
    # Filter predictions with confidence above threshold
    best_estimates = pr_errors[confidence_est >= threshold]

    # Compute mean absolute error on high-confidence predictions
    return np.mean(np.abs(best_estimates))

def Evaluate():
    """
    Top-level function evaluation function.

    Runs the pulse rate algorithm on the Troika dataset and returns an aggregate error m

    Returns:
        Pulse rate error on the Troika dataset. See AggregateErrorMetric.
    """
    data_fls, ref_fls = LoadTroikaDataset()
    errs, confs = [], []
    for data_fl, ref_fl in zip(data_fls, ref_fls):
        errors, confidence, *_ = RunPulseRateAlgorithm(data_fl, ref_fl)
        errs.append(errors)
        confs.append(confidence)
    errs = np.hstack(np.array(errs))
    confs = np.hstack(np.array(confs))
    return AggregateErrorMetric(errs, confs)

Evaluate()
```